

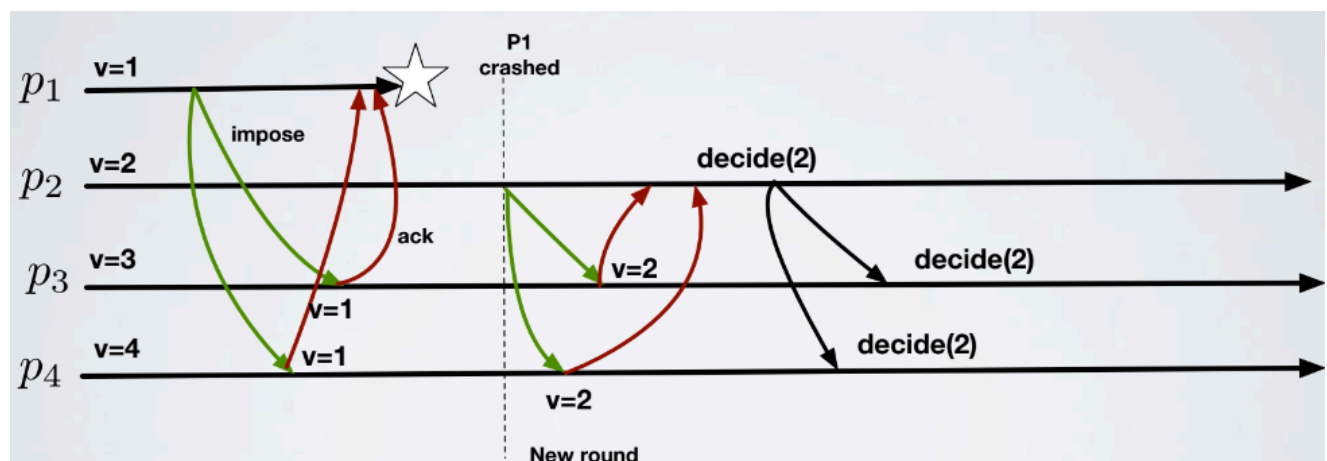
Distributed Systems - 10_12

We will only see two types of paradigms for **Byzantine Failures**, broadcast and consensus. In crash-stop failures, the process simply stops to work. In a byzantine failure, however, a process might start acting differently from how it was intended, stopping to follow the algorithm. They behave in an arbitrary way. It might seem that they are following the algorithm, but at a certain random point they stop. Remember that $\text{Crash-Stop} \subset \text{Byzantine}$, so if an algorithm is able to manage Byzantine failures, it will be able to handle crash-stop failures as well.

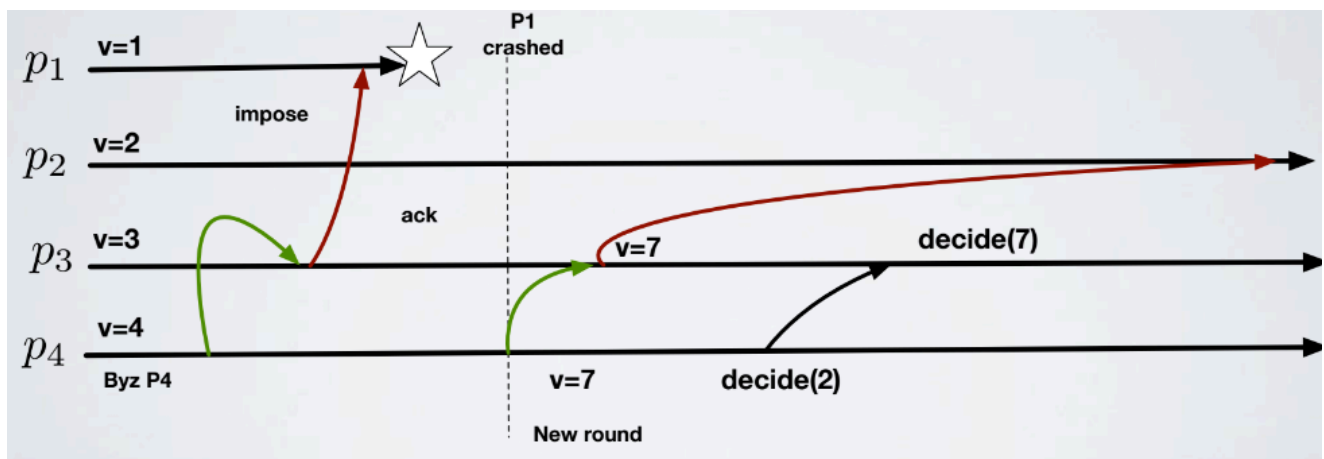
The number of types of actions due to a Byzantine failure is basically infinite, so when we build an algorithm that deals with Byzantine failures, it has to deal with all the possibilities. Asynchrony is also under control of the byzantines, as a third party could control the scheduler and delays of channels. However, we assume the fairness of the scheduler is kept.

Now, let us imagine we have this run for a hierarchical consensus. If we do not prevent the byzantine failure of having a process sending messages faking an arbitrary sender.

Normal execution:



Byzantine execution:



Here p_4 is imposing himself to p_3 (probably faking by saying it's p_1).

If a byzantine can impersonate other processes, non-trivial computations are impossible. Thus, the need of having **authentication**.

We'll do this with Cryptography. We'll need some mechanisms:

- Hash functions
- Message Authentication Codes:
 - We'll have a distributed oracle Auth
 - Authenticate operation: any sender process p will invoke the oracle sending message m and sender q , and will receive a special object a .
 - Verifyauth operation: Any receiving process q will invoke the oracle with message m , sender p and object a . The oracle will answer yes $\iff p$ previously invoked oracle call on message m with sender q , receiving a .
- Public signatures:
 - There is a distributed oracle Sign.
 - Sign operation: Any sender p invokes the oracle with m receiving s .
 - VerifySign operation: Any process q invokes the oracle with m , p and s . VerifySign answers yes $\iff p$ previously invoked the oracle call with message m and received s .

The oracle is a library located inside each process, having two functions, one to create a mac and one to verify it. For public signatures, we'll have a oracle as well, but it's more powerful, as it proves if we have received a certain signature from a certain person.

So, here are the properties of the **Authenticated Perfect link**:

- (Reliable delivery): If a correct process sends a message m to a correct process q , then q eventually delivers m .
- (No duplication): No message is delivered by a **correct** process more than once.

- (Authenticity): If some correct process q delivers a message m with sender p , and p is correct, then m was previously sent to q by p .

Note: **correct** highlights the difference with the properties of the p_2p link.

In the authenticity, we say that p has to be correct, otherwise we could have a case in which p_2 (correct) receives a message from p_4 (byzantine). But there could be another byzantine process, like p_3 , asking p_4 for its mac, and then sending a message to p_2 faking to be p_4 .

Here is the code:

Algorithm 2.4: Authenticate and Filter

Implements:

AuthPerfectPointToPointLinks, **instance** al .

Uses:

StubbornPointToPointLinks, **instance** sl .

upon event $\langle al, Init \rangle$ **do**
 $delivered := \emptyset$;

upon event $\langle al, Send \mid q, m \rangle$ **do**
 $a := authenticate(self, q, m)$;
trigger $\langle sl, Send \mid q, [m, a] \rangle$;

upon event $\langle sl, Deliver \mid p, [m, a] \rangle$ **do**
if $verifyauth(self, p, m, a) \wedge m \notin delivered$ **then**
 $delivered := delivered \cup \{m\}$;
trigger $\langle al, Deliver \mid p, m \rangle$;

So, when someone wants to send a message, we authenticate it via the oracle, and only then we will send the message. Since the link is stubborn, an infinite number of messages will be sent. When a process finally receives the message, he will run the oracle to verify the value of a , checking if the message has already been delivered, only if not he will put it in the delivered set.

There are some properties which are meaningful in byzantine failures, for example UC4:

- Uniform Agreement (UC4): No two processes decide differently.

This property can't be implemented with byzantine failures, as a byzantine process could choose to do whatever, even deciding a different value (note that we are not specifying "correct"). Thus no uniform property holds when byzantines are used.

Let's now take a look at **Byzantine Broadcast**.

With crash failures, the problem in a broadcast was located in the `for` loop, such that the process could crash while sending messages to other processes. We fixed this problem by introducing the reliable broadcast with the "all or nothing" primitive. This won't work as is with byzantine failures, as a byzantine process can choose to send different messages to different people while not crashing. We have to implement this "all or nothing" semantic while keeping the run consistent.

We want that honest processes are not able to deliver different messages sent by the byzantine in a run of broadcast.

- Step 1: We build a Broadcast primitive (Consistent Broadcast), such that if we have a byzantine broadcaster either no correct processes deliver any messages, or if a correct process deliver m , then no other corrects deliver a message different from m .
- Step 2: We build a more powerful Broadcast primitive (Byzantine Reliable Broadcast), in which we have the "all or nothing" semantic.

In the consistent broadcast, byzantine failure will behave as crash-stop.

Here is the properties of **Consistent Broadcast**:

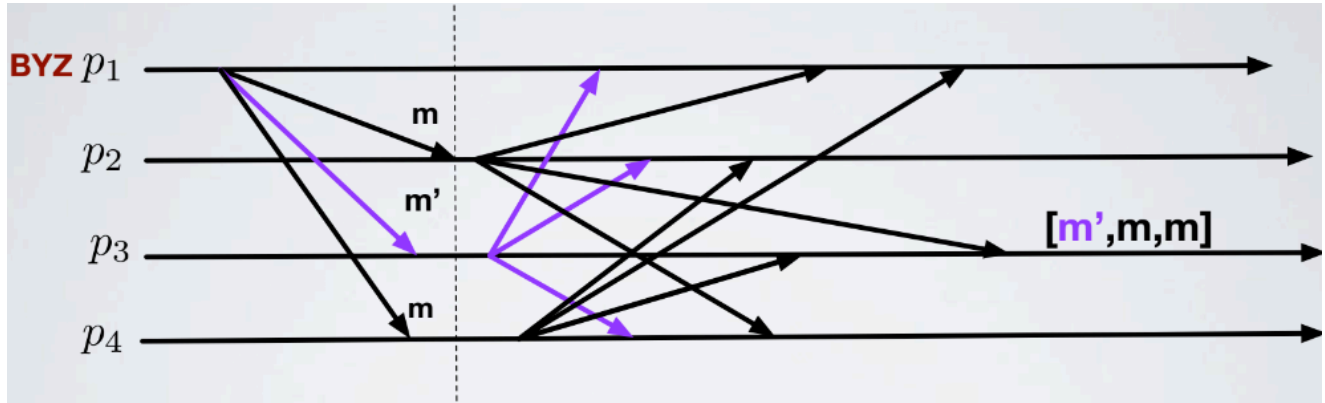
- (Validity): If a correct process p broadcasts a message m , then every correct process eventually delivers m .
- (No duplication): Every correct process delivers at most one message.
- (Integrity): If some correct process delivers a message m from process p , and p is correct, then m was previously broadcasted by p .
- (Consistency): If some correct process delivers a message m and another correct process delivers message m' , then $m = m'$.

Note: the instance of this algorithm is one-shot, thus it refers only to the sending of one message (if we want to send a message m_j after m_i , we have to make a new instance).

Also, we have to have that $N > 3f$.

So, how do we actually fix the fact that a byzantine process might, for example, send m to p_2 and p_4 , and $m' \neq m$ to p_3 ? We could have each process broadcast

again the message (having one echo). Is this enough however?



Do we need two echoes? Three echoes?

Let's say that p_1 , seeing that p_3 has still to decide, echoes m' to him. p_3 will then see locally two votes for m and two votes for m' . If the byzantine process plays it well, it could create total confusion.

Even worse, the byzantine could echo m' to p_3 , and m to p_2 , leading to those two having two different "decided" messages ("decided" as in deciding the correct message of the broadcast, not the modified version, so that everyone has the same).

Why not wait for all echoes (four)? If the byzantine process decides to remain silent, other processes will never reach the minimum, thus stalling and breaking liveness.

The solution is in the use of quorums. We define a value k that is a threshold of how many echoes we have to see. We could consider an echo as a vote, such that an honest process gives the same vote for everyone, while a byzantine could not. Suppose p delivers message m reaching threshold k . Then it means there is a set V_m of k processes who voted for m . Now suppose p' delivers message m' reaching threshold k , thus having a set $V_{m'}$ of k processes who voted m' .

If a process is byzantine, then he can find himself in $V_m \cap V_{m'}$ by lying. However, if a process is honest, then he is in $V_m \cap V_{m'}$ only if he voted the same thing, thus having $m = m'$. If inside the intersection there are $f + 1$ processes, then we can be sure that at least one is honest, and this grants us that $m = m'$.

$$k > \frac{N + f}{2}$$

This quantity is known as byzantine quorum.

Now, let's take a look at the code for **byzantine consistent broadcast**:

Algorithm 3.16: Authenticated Echo Broadcast

Implements:
 ByzantineConsistentBroadcast, **instance** *bcb*, with sender *s*.

Uses:
 AuthPerfectPointToPointLinks, **instance** *al*.

```

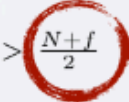
upon event  $\langle bcb, \text{Init} \rangle$  do
    sentecho := FALSE;
    delivered := FALSE;
    echos :=  $\perp^N$ ;

upon event  $\langle bcb, \text{Broadcast} \mid m \rangle$  do                                // only process s
    forall  $q \in \Pi$  do
        trigger  $\langle al, \text{Send} \mid q, [\text{SEND}, m] \rangle$ ;

upon event  $\langle al, \text{Deliver} \mid p, [\text{SEND}, m] \rangle$  such that  $p = s$  and sentecho = FALSE do
    sentecho := TRUE;
    forall  $q \in \Pi$  do
        trigger  $\langle al, \text{Send} \mid q, [\text{ECHO}, m] \rangle$ ;

upon event  $\langle al, \text{Deliver} \mid p, [\text{ECHO}, m] \rangle$  do
    if echos[p] =  $\perp$  then
        echos[p] := m;

upon exists  $m \neq \perp$  such that  $\#(\{p \in \Pi \mid echos[p] = m\}) > \frac{N+f}{2}$ 
    and delivered = FALSE do
        delivered := TRUE;
        trigger  $\langle bcb, \text{Deliver} \mid s, m \rangle$ ;
    
```



Correctness is guaranteed
as long as $N > 3f$

The last handler basically tells us to deliver with Byzantine Consistent Broadcast if we have a number of processes echoing a certain message that is $> \frac{N+f}{2}$.

Let's now see the proof, starting from validity.

If the sender is correct, at least $N - f$ processes (the correct ones) echoes message *m*. In order for others to deliver *m* we need $N - f > (N + f)/2$, that is $\frac{N}{2} > f + \frac{f}{2}$, that is $N > 3f$.

No duplication: immediate from authenticated channel properties.

Integrity: immediate from authenticated channel properties.

It remains to prove consistency.

A process delivers *m* only after al-delivering more than $\frac{N+f}{2}$ ECHO messages (he has to receive *k* echo messages containing message *m* before being able to deliver it). $\frac{N+f}{2}$ represents a byzantine quorum. Any two byzantine quorums overlaps in at least one correct process. In fact, the intersection of any two byzantine quorums is at least $f + 1$, and any set of $f + 1$ processes contains at least one correct.

Basically, if we have a set of cardinality $f + 1$ we can be sure that at least one process will be correct, as we have at max f byzantine processes, so the last one is necessarily correct (if we count, even in the worst case, the first f as all

byzantine).

Now, let p deliver message m , so it has received a byzantine quorum for it. Then, let p' deliver m' , then it has received a byzantine quorum, not necessarily the one above. Because two quorums intersect in at least one value, we can be sure that at least one process is correct, and if that process appears in both quorums, then we can be sure $m = m'$.

The performances:

- Messages: at most n^2 , counting only messages generated by correct processes (as byzantines could potentially send an infinite amount of messages).
- Delay: 2 steps.